```
BEGIN;
--
-- Create model Question
--
CREATE TABLE "polls_question" (
    "id" serial NOT NULL PRIMARY KEY,
    "question_text" varchar(200) NOT NULL,
    "pub_date" timestamp with time zone NOT NULL
);
--
-- Create model Choice
--
CREATE TABLE "polls_choice" (
    "id" serial NOT NULL PRIMARY KEY,
    "choice_text" varchar(200) NOT NULL,
    "votes" integer NOT NULL,
    "question_id" integer NOT NULL
);
ALTER TABLE "polls_choice"
  ADD CONSTRAINT
"polls_choice_question_id_c5b4b260_fk_polls_question_id"
    FOREIGN KEY ("question_id")
    REFERENCES "polls_question" ("id")
    DEFERRABLE INITIALLY DEFERRED;
CREATE INDEX "polls_choice_question_id_c5b4b260" ON
"polls_choice" ("question_id");

COMMIT;
```

project. Here, the developer need not delete any table or database or start from a scratch. It has a specialisation to upgrade your DB in a real-time environment with no loss of data. Furthermore, here is the three step guide for the alteration of the model:

1. Step 1: Alter model in 'models.py'.
2. Step 2: Run the 'python manage.py makemigrations' for the creation of altered migrations.
3. Step 3: Run command 'python manage.py migrate' for the application of the changes in the database.

Here is a reason why we are using separate commands for making and applying migrations. It is because whenever we commit migrations to our 'version control systems' and thereafter ship them with the specific app; it makes the development far easier. Additionally, they can be used by other developers and in the production work as well.